

تقرير فحص شامل — بدون أي تعديل على الكود

نظام Nexus ERP

مراجعة كاملة لاكتمال النظام مقابل متطلبات نظام ERP حقيقي — تغطي كل الوحدات (المحاسبة، المبيعات، المشتريات، المخزون، نقطة البيع، الموارد البشرية، الأمان، البنية التحتية) والقطاعات الأربعة (السياحة، الضيافة، تأجير السيارات، التوظيف).

16

منخفض

25

متوسط

15

عالي

12

حرج

إجمالي الملاحظات: 68 | تاريخ التقرير: 17 أغسطس 2026

ما تم التأكد أنه يعمل بشكل حقيقي وسليم

الفحص لم يكن بحثًا عن مشاكل فقط — كل نقطة هنا تم التحقق منها فعليًا في الكود وليست افتراضيًا.

- شبكة الثقة (Trust Network) حقيقية فعليًا عبر التينانتس (hashing + k-anonymity)، وليست مجرد اسم.
- الشحن: تكامل حقيقي وقوي مع شركات الشحن (Bosta/Mylerz) عبر webhooks ومربوط صح بالفواتير.
- الفوترة والاشتراكات: باقات حقيقية، بوابة دفع Paymob مربوطة فعليًا بمفاتيح حقيقية، ونظام متابعة المتأخرين (dunning) تم إصلاحه مؤخرًا.
- السوق (Marketplace): تفعيل القطاعات إنفاذ حقيقي على مستوى الـ API (require_plugin)، وليس شكليًا فقط (لكن القائمة الجانبية لا تعكس ذلك — انظر القسم الأول).
- الفندق: تقويم الحجوزات، تسجيل الدخول/الخروج، والفوترة (folio) كلها شغالة وحقيقية.
- تأجير السيارات: إدارة الأسطول، دورة حياة العقد، التسعير، والفحص المزدوج كلها شغالة وحقيقية.
- التوظيف: أوامر التوظيف، سير العمل المرحلي، والعمولة المرحلة تلقائيًا للحسابات كلها شغالة وحقيقية.
- اختبارات العزل بين المستأجرين (isolation tests) تعمل فعليًا داخل CI على قواعد بيانات حقيقية.
- تسجيل الخروج يُبطل رمز التحديث فعليًا من Redis (وليس شكليًا).
- الإشعارات عبر واتساب (Meta Graph API الحقيقي) والبريد الإلكتروني الاحتياطي تعمل فعليًا.
- الحماية من حقن (SQL injection) قوية فعليًا — كل الاستعلامات تقريبًا تمر عبر آلية أمانة موحدة، ولا توجد ثغرات XSS في أي مكان بالواجهة.
- إعدادات CORS مضبوطة بشكل احترافي، مع فحص يمنع تشغيل النظام في الإنتاج بإعدادات أمان افتراضية غير آمنة.
- معظم الـ migrations (28 من 31) فيها كود تراجع (downgrade) حقيقي وليس فارغًا، والأرقام المالية كلها من نوع Decimal دقيق وليس float في كل الوحدات التي تم فحصها.
- نظام المراقبة والتسجيل (logging/Sentry/OpenTelemetry/health checks) مبني بشكل احترافي وحقيقي في الباك إند.

1. الأمان وحسابات المستخدمين

تسجيل الدخول، استعادة الحساب، الجلسات، الصلاحيات

حرج استعادة كلمة المرور غير موجودة نهائيًا

المشكلة رابط "نسيت كلمة المرور؟" في صفحة الدخول مجرد href="#" لا يفعل شيئًا. لا توجد أي صفحة (forgot-password / reset-password) في الواجهة الأمامية، ولا يوجد أي endpoint في الباك إند لإصدار أو التحقق من رمز إعادة التعيين. أي مستخدم ينسى كلمة مروره يفقد الوصول لحسابه نهائيًا بدون تدخل يدوي من مسؤول قاعدة البيانات مباشرة. في نظام حقيقي يستخدمه عملاء، هذا يعني تذاكر دعم فني لا نهائية أو حسابات مهجورة.

الحل المقترح إضافة endpoint في الباك إند لطلب إعادة تعيين (يرسل رابط/رمز عبر البريد الإلكتروني، مع صلاحية زمنية قصيرة ومحاولات محدودة)، وصفحتين في الواجهة (طلب الاستعادة + إدخال كلمة مرور جديدة). البنية التحتية للـ SMTP موجودة وتعمل بالفعل، فقط تحتاج ربط بهذا التدفق.

لا يوجد تدوير (rotation) لرمز التحديث (refresh token) عالي

المشكلة endpoint الخاص بـ auth/refresh يسمح باستخدام نفس رمز التحديث عددًا غير محدود من المرات طوال مدة صلاحيته (7 أيام) دون إبطاله أو استبداله بعد أول استخدام.

التأثير لو تسرب رمز تحديث واحد (من جهاز مسروق، أو من سجلات، أو هجوم XSS)، يبقى صالحًا للاستخدام الكامل لمدة 7 أيام كاملة بغض النظر عن عدد مرات استخدامه من قبل المهاجم أو صاحب الحساب الأصلي، مع عدم وجود أي طريقة لاكتشاف أن هناك استخدامًا مزدوجًا.

الحل المقترح تطبيق نمط "rotate on use": عند كل استخدام لرمز تحديث، يتم إبطاله فورًا وإصدار رمز جديد بدلاً منه. لو حاول أي طرف استخدام رمز تم إبطاله بالفعل (يعني تم استخدامه مرتين)، يتم اعتبار ذلك مؤشر اختراق وإبطال كل جلسات المستخدم فورًا.

لا توجد مصادقة ثنائية (2FA) للمستخدمين الداخليين متوسط

المشكلة لا يوجد أي خيار لتفعيل 2FA لحسابات الموظفين/الإدارة. الموجود فقط هو تدفق OTP وهمي (mock) غير مكتمل التحقق مخصص لعملاء البوابة الخارجية فقط.

التأثير حسابات الإدارة والمحاسبة (الأكثر حساسية في النظام) محمية بكلمة مرور فقط، وهو أضعف مستوى حماية ممكن لنظام يدير بيانات مالية.

الحل المقترح إضافة 2FA اختيارية (TOTP عبر تطبيق مثل Google Authenticator) للحسابات ذات الصلاحيات العالية (OWNER/ADMIN) كحد أدنى، مع إمكانية جعلها إلزامية لاحقًا.

خانة "تذكرني" غير فعّالة منخفض

المشكلة الخانة موجودة في نموذج تسجيل الدخول ومسجلة في الحالة (state)، لكنها لا تُقرأ ولا تُرسل في onSubmit — تصرفها الفعلي معدوم تمامًا.

التأثير تجربة مستخدم مزلزلة: المستخدم يظن أنه اختار البقاء مسجلًا دخوله وهو ليس كذلك.

الحل المقترح إما تفعيلها فعليًا (بتمديد صلاحية رمز التحديث عند تفعيلها)، أو إزالتها من الواجهة حتى لا تعطي انطباعًا خاطئًا.

الصلاحيات (RBAC) — أكبر فجوة أمان في النظام: الأدوار موجودة بالاسم فقط ولا تُفرض عمليًا حرج

المشكلة قاعدة الكود تعرّف 5 أدوار فعليًا (OWNER, ADMIN, STAFF, SALES, ACCOUNTING) — يعني النموذج نفسه فيه "محاسب" و"مبيعات" كأدوار معرّفة، وليس فقط OWNER/ADMIN. لكن هذا التعريف غير مُفَعَّل عمليًا في أي مكان: لا يوجد فحص للدور في الـ API لمعظم الشاشات (التفرقة الفعلية تكاد تقتصر على OWNER/ADMIN مقابل الباقي)، ولا توجد أي فلترة في الواجهة نفسها — القائمة الجانبية والصفحات متطابقة تمامًا لكل مستخدم أيًا كان دوره. كذلك لا يوجد دور "كاشير" منفصل خالص في النموذج أصلًا رغم وجود شاشة نقطة بيع كاملة.

التأثير أي موظف يسجل دخوله — سواء كاشير أو مبيعات أو محاسب مبتدئ — يرى نفس شاشات الإدارة والإعدادات الحساسة بالضبط التي يراها المالك. هذه فجوة تمنع أي شركة حقيقية من توزيع النظام على أكثر من شخص بأمان: لا يمكن إعطاء كاشير وصولاً لنقطة البيع فقط، ولا محاسب وصولاً للحسابات فقط دون باقي النظام.

الحل المقترح خطوتان متلازمتان: (1) تفعيل فحص الدور فعليًا على مستوى كل endpoint حساس في الـ API بدل الاكتفاء بالتحقق من تسجيل الدخول فقط. (2) فلترة عناصر القائمة الجانبية والصفحات في الواجهة حسب دور المستخدم الفعلي بدل عرض كل شيء للجميع. يُفَضَّل أيضًا إضافة دور "كاشير" مخصص محدود الصلاحيات على نقطة البيع فقط، بما أن هذا الدور غائب تمامًا عن النموذج الحالي.

القائمة الجانبية لا تُفلتر حسب القطاعات المفصلة للمستأجر

عالي

المشكلة

عند التسجيل، المستخدم يختار القطاعات التي يعمل بها (مثلاً "الضيافة والفنادق" فقط). لكن كود القائمة الجانبية (dashboard/layout.tsx) مبني كمصفوفة ثابتة (navSections) تحتوي كل الأقسام والقطاعات الأربعة معاً (السياحة، الضيافة، تأجير السيارات، التوظيف) بلا أي شرط يتحقق من القطاعات المفصلة فعلياً لحساب المستأجر.

التأثير

أي مستخدم يرى في القائمة الجانبية روابط لقطاعات لم يختَرها أصلاً ولم يدفع/يفعلها، ويمكنه الدخول لصفحاتها. الحماية الحقيقية موجودة فقط في الباك إند (require_plugin) يمنع فعلياً جلب بيانات القطاع غير المفعل، لكن تجربة المستخدم مربكة: يرى رابطاً يبدو متاحاً ثم يفاجأ برسالة خطأ صلاحية عند محاولة استخدامه، بدل أن يختفي الرابط من الأساس.

الحل المقترح

بناء القائمة الجانبية ديناميكياً بناءً على قائمة القطاعات المفصلة الفعلية لكل مستأجر (نفس البيانات التي تُستخدم في صفحة السوق/marketplace لتحديد ما هو مفعل)، بحيث لا يظهر أي رابط لقطاع غير مفعل من الأساس بدل الاعتماد على رفض ال API فقط.

2. المحاسبة والمالية

دفتر الأستاذ، التقارير المالية، الشيكات، الفاتورة الإلكترونية

لا توجد تقارير مالية متاحة في الواجهة (ميزان مراجعة / قائمة دخل / ميزانية)

حرج

المشكلة

كود حساب ميزان المراجعة والتقارير المالية موجود فعلياً في الباك إند (accounting/reports/trial_balance.py) لكنه غير مربوط بأي API، وواجهة المحاسبة تحتوي فقط على تابين: القيود اليومية ودليل الحسابات.

التأثير

لا يمكن لأي محاسب إغلاق الشهر أو مراجعة الأرصدة أو استخراج قائمة دخل من داخل النظام؛ هذه من أبسط متطلبات أي ERP محاسبي.

الحل المقترح

كشف endpoint لكل تقرير جاهز في الباك إند، وبناء 2-3 صفحات في الواجهة (ميزان المراجعة، قائمة الدخل، الميزانية العمومية) مع إمكانية اختيار الفترة الزمنية والتصدير ك PDF/Excel.

زر إرسال الفاتورة الإلكترونية لمصلحة الضرائب (ETA) غير موجود في الواجهة

حرج

المشكلة

خط الأنابيب الكامل (بناء المستند، التوقيع، الإرسال، استقبال الإيصـال) موجود وجاهز في الباك إند بالكامل، لكن لا يوجد أي زر أو مسار في الواجهة يستدعيه — المستخدم لا يستطيع إرسال أي فاتورة إلكترونية فعلياً من الشاشة.

التأثير

هذا يعطل الالتزام الضريبي الإلزامي في مصر بالكامل رغم أن كل الشغل التقني منجز فعلاً — أولوية قصوى لأنها فجوة "جاهز لكن مقفول" وليست فجوة تطوير حقيقية.

الحل المقترح

إضافة زر "إرسال للـ ETA" في شاشة الفاتورة (فردية وجماعية)، وعرض حالة الإرسال (مقبولة/مرفوضة/معلقة) وأسباب الرفض (validation_errors) بوضوح في نافذة التفاصيل — الحقول موجودة بالفعل في الباك إند، فقط تحتاج عرضاً.

ارتداد الشيك لا يُحدث أي أثر مالي أو تنبيه فعلي

عالي

المشكلة

دالة bounce_cheque () تكتفي بتغيير حالة الشيك فقط. لا يوجد عكس تلقائي للقيد المحاسبي المرتبط، ولا أي علامة على سجل العميل، ولا تقويم لتذكير بمواعيد استحقاق الشيكات.

التأثير

ارتداد شيك عميل يمر بصمت محاسبياً — الأرصدة المالية تبقى كأن شيئاً لم يحدث، والفريق لا يُنبّه لمتابعة العميل.

الحل المقترح

ربط bounce_cheque () بعكس تلقائي للقيد الأصلي (أو قيد تصحيحي)، ووضع علامة على ملف العميل، وإضافة تقويم قائمة لمواعيد استحقاق الشيكات القادمة مع تنبيهات.

3. المبيعات والمشتريات والمخزون

متوسط لا يوجد "أمر بيع" (Sales Order) — الفاتورة تُنشأ مباشرة

المشكلة دورة المبيعات تخطى مرحلة أمر البيع بالكامل وتذهب مباشرة لإنشاء فاتورة، رغم أن هذه مرحلة أساسية في أي دورة مبيعات B2B حقيقية (عرض سعر → أمر بيع → فاتورة → تحصيل).

التأثير لا يمكن تتبع الطلبات المعلقة قبل الفاتورة، ولا إدارة الحجز المسبق للمخزون بناءً على أوامر مؤكدة لم تُفوتر بعد.

الحل المقترح إضافة نموذج ومسار SalesOrder قبل الفاتورة، مع إمكانية تحويل أمر البيع المعتمد لفاتورة بضغطة واحدة.

عالي لا توجد واجهة لتسجيل دفعات على الفواتير، ولا مرتجعات مبيعات

المشكلة لا يوجد أي مسار في الواجهة لتسجيل دفعة (Payment) مقابل فاتورة، ولا لعمل مرتجع بضاعة أو إشعار دائن.

التأثير لا يمكن للنظام إدارة التحصيل الفعلي من العملاء أو معالجة المرتجعات — من أهم الوظائف اليومية لأي نظام محاسبي/مبيعات.

الحل المقترح بناء شاشة تسجيل دفعة (كاملة/جزئية) مرتبطة بالفاتورة وبقيد محاسبي تلقائي، وشاشة مرتجع مبيعات (RMA) تُنشئ إشعار دائن وتُرجع الكمية للمخزون.

عالي لا يوجد أمر شراء (PO) ولا إذن استلام بضاعة — و"three-way match" شكلي فقط

المشكلة فواتير الموردين تُنشأ مباشرة بدون أمر شراء مسبق أو إذن استلام مرتبط، لذلك شارة "three-way match" الظاهرة في الجدول لا تقارن بيانات حقيقية (لا يوجد شيء تقارنه به فعليًا).

التأثير لا رقابة مسبقة على المشتريات (اعتماد قبل الشراء)، ولا تتبع للفرق بين ما طُلب وما وُرد وما فُوتر — وهي وظيفة رقابة مالية أساسية.

الحل المقترح إضافة نموذجي PurchaseOrder وGoodsReceipt، وربط الفاتورة الفعلية بهما بحيث تصبح شارة المطابقة الثلاثية ذات معنى حقيقي (كمية الطلب مقابل كمية الاستلام مقابل كمية الفاتورة).

متوسط لا يوجد تقرير تقييم مخزون، ولا اقتراح إعادة طلب، ولا جرد فعلي

المشكلة هذه الوظائف الثلاث غير متاحة في أي مكان بالواجهة رغم أن حسابات التكلفة (landed cost) موجودة بالباك إند لأجزاء أخرى من النظام.

التأثير لا يمكن معرفة قيمة المخزون الحالية بسهولة، ولا يوجد تنبيه عند اقتراب نفاد صنف، ولا طريقة رسمية لمطابقة الجرد الفعلي بالمخزون الدفتر.

الحل المقترح إضافة 3 شاشات: تقرير تقييم المخزون (بالتكلفة الحالية)، لوحة اقتراحات إعادة الطلب (بناءً على حد أدنى/معدل استهلاك)، وشاشة جرد دوري (stock take) تُنشئ تسويات تلقائية عند الفروقات.

منخفض لا توجد واجهة لتحويلات المخزون بين الفروع/المخازن

المشكلة لا يوجد مسار في الواجهة لنقل كمية من مخزن لآخر.

التأثير أي نظام متعدد الفروع يحتاج هذه الوظيفة كأساسيات، وغيابها يعني تسجيل التحويلات يدويًا خارج النظام.

الحل المقترح إضافة شاشة تحويل مخزون بسيطة (من مخزن، إلى مخزن، صنف، كمية) تُنشئ حركتين متطابقتين في المخزونين.

4. نقطة البيع (POS)

عالي لا يوجد استرجاع/مرتجع من نقطة البيع

المشكلة

لا يوجد أي endpoint أو زر لعمل مرتجع أو استرداد لعملية بيع تمت بالفعل من الكاشير. أي طلب استرجاع من عميل يحتاج تدخلاً يدوياً خارج النظام بالكامل، وهو أمر يومي في أي محل بيع بالتجزئة.

التأثير

الحل المقترح

إضافة مسار استرجاع (كامل/جزئي) من واجهة نقطة البيع، يعكس الكمية للمخزون والمبلغ من الصندوق ويُنشئ القيد المحاسبي العكسي تلقائياً.

متوسط طريقة الدفع في POS مثبتة على "نقدي" فقط

المشكلة

الواجهة الأمامية ترسل payment_method: "cash" دائماً بشكل ثابت في الكود، رغم أن الباك إند يدعم طرق دفع أخرى. لا يمكن تسجيل بيع بالبطاقة أو التحويل من نقطة البيع، رغم أن النظام الخلفي جاهز لذلك.

التأثير

الحل المقترح

إضافة اختيار طريقة الدفع في شاشة الكاشير (نقدي/بطاقة/محفظة) وإرسال القيمة الفعلية المختارة بدل القيمة الثابتة.

منخفض لا توجد حاسبة للباقي (change due) ولا دعم لقارئ الباركود

المشكلة

لا حساب تلقائي للباقي عند إدخال المبلغ المدفوع، ولا معالجة خاصة لإدخال الباركود (سكانر). إبطاء عملية البيع اليومية وزيادة احتمال خطأ بشري في حساب الباقي يدوياً.

التأثير

الحل المقترح

إضافة حقل "المبلغ المستلم" مع حساب الباقي تلقائياً، ودعم قراءة الباركود كإدخال سريع للصنف بدل البحث اليدوي.

5. جهات الاتصال، الموردون، شبكة الثقة

حرج كود ميت وخطر في وحدة جهات الاتصال (contacts/service.py)

المشكلة

هذا الملف يفترض وجود حقول (address, tags, metadata) ونموذج (ContactRelationship) غير موجودة فعلياً في قاعدة البيانات — لو استدعي هذا الكود من أي مكان سيتسبب في كراش فوري.

التأثير

قنبلة موقوتة: أي تعديل مستقبلي يستدعي هذه الدالة عن طريق الخطأ سيكسر النظام في الإنتاج مباشرة.

الحل المقترح

إما حذف الكود الميت بالكامل، أو تحديثه فعلياً ليطابق النموذج الحقيقي الحالي إن كانت هذه الحقول مطلوبة فعلاً — ثم تغطيته باختبار يمنع تكرار المشكلة.

عالي مفهومان منفصلان لـ "المورد" بدون ربط بينهما

المشكلة

يوجد Contact عام بنوع SUPPLIER من جهة، و Vendor/VendorRateCard خاص بوحدة الحالات (المستخدم في السياحة/التوظيف) من جهة أخرى، بدون أي مفتاح ربط (FK) بينهما.

التأثير

خطر حقيقي لتكرار نفس المورد كسجلين منفصلين تماماً، مع بيانات قد تتضارب بين النسختين.

الحل المقترح

إضافة ربط (FK) من Vendor إلى Contact، أو توحيد المفهومين في نموذج واحد بأدوار متعددة، مع فحص تكرار عند الإنشاء (بالاسم/رقم الهاتف/الرقم الضريبي).

متوسط لا يوجد فحص تكرار عند إنشاء جهة اتصال جديدة

المشكلة

لا يتحقق النظام مما إذا كانت جهة الاتصال (بنفس الاسم/الهاتف) موجودة بالفعل قبل إنشاء سجل جديد. تراكم سجلات مكررة لنفس العميل/المورد مع مرور الوقت، مما يصعب التقارير والتواصل.

التأثير

الحل المقترح

إضافة فحص تشابه بسيط (بالهاتف أو البريد) عند الإنشاء، مع تنبيه "هل تقصد هذا السجل الموجود؟" قبل التأكيد.

شبكة الثقة: السر الخاص بالتشفير placeholder، ومنع تكرار الاستعلام غير مُفعّل

متوسط

المشكلة `get_vault_pepper()` يستخدم قيمة نصية ثابتة بدل إدارة أسرار حقيقية، وآلية الحد من تكرار الاستعلامات (rate-limiting) لمنع استكشاف الأرقام (enumeration) موجودة كدالة فارغة (stub) غير مفعّلة. رغم أن آلية الخصوصية نفسها (k-anonymity + hashing) مصممة جيدًا وحقيقية، هذان الجزآن يضعفان الحماية الفعلية إذا تعرض النظام لمحاولة استغلال منظمة.

التأثير

الحل المقترح نقل الـ pepper لنظام إدارة أسرار حقيقي (متغير بيئة محمي/vault خارجي)، وتفعيل الحد من معدل الاستعلامات فعليًا بدل تركه stub.

6. الشحن والتسويات والاستيراد

خطأ حقيقي: مصاريف الاستيراد لا تصل أبدًا لتكلفة المخزون

حرج

المشكلة جدول `DossierCostLayer` (الذي يفترض أن يربط مصاريف الاستيراد بتكلفة الأصناف) يُقرأ فقط في الكود ولا يُكتب فيه أبدًا من أي مكان. دالة `close_dossier` تُغلق الملف دون توزيع أي تكلفة فعليًا.

التأثير تكلفة الأصناف المستوردة في النظام غير دقيقة فعليًا — كل مصاريف الشحن/الجمارك/التخليص المُدخلة تُحسب لكنها لا تنعكس أبدًا على سعر تكلفة الصنف الحقيقي، ما يؤثر على دقة هامش الربح وتقييم المخزون في كل مكان يعتمد عليها.

الحل المقترح إصلاح دالة `close_dossier` (أو الدالة المسؤولة عن توزيع التكلفة) لتنشئ فعليًا سجلات `DossierCostLayer` حسب طريقة التوزيع المطلوبة (بالوزن/بالقيمة/بالكمية) عند إغلاق ملف الاستيراد.

التسويات (Settlements): منطق قوي بالباك إند لكن بدون أي واجهة

عالي

المشكلة منطق المطابقة التلقائي بين كشوف الحسابات والمعاملات موجود ومتطور، لكن لا توجد واجهة رفع ملفات ولا شاشة مراجعة/اعتراض على الفروقات.

التأثير فريق المحاسبة لا يستطيع فعليًا استخدام هذه القدرة القوية لأنها غير مرئية بالكامل من الواجهة.

الحل المقترح إضافة شاشة رفع كشف حساب (CSV/Excel)، وعرض نتائج المطابقة التلقائي، وواجهة لمعالجة السطور غير المتطابقة يدويًا.

لا توجد إدارة مستندات في وحدة الاستيراد

متوسط

المشكلة لا يوجد مكان لرفع أو حفظ بوليصة الشحن (Bill of Lading) أو البيان الجمركي أو أي مستندات استيراد رسمية. المستندات الرسمية المطلوبة قانونيًا لكل شحنة مستوردة تُدار بالكامل خارج النظام.

التأثير

الحل المقترح إضافة قسم مرفقات لكل ملف استيراد (dossier) يسمح برفع وتصنيف المستندات المرتبطة به.

7. الموارد البشرية، فريق العمل، الحالات (Cases)

لا يوجد ربط بين حساب "فريق العمل" (تسجيل الدخول) وسجل "الموظف" في HR

عالي

المشكلة نموذج `system.User` (حسابات الدخول والصلاحيات) ونموذج `hr.Employee` (سجل الراتب) منفصلان تمامًا بدون مفتاح ربط بينهما.

التأثير يمكن أن يوجد نفس الشخص كسجلين منفصلين تمامًا (حساب دخول من جهة، وموظف براتب من جهة أخرى) بدون أي طريقة لمعرفة أنهما نفس الشخص، مما يعقّد أي تقرير يربط الأداء بالراتب أو الصلاحيات بالوظيفة.

الحل المقترح إضافة حقل ربط اختياري (`employee_id`) في نموذج `User` يشير لسجل HR المقابل، مع واجهة لربط الحسابات الموجودة يدويًا أولاً ثم اشتراط الربط عند إنشاء مستخدمين جدد.

متوسط لا توجد إدارة إجازات أو حضور وانصراف

المشكلة	وحدة HR تغطي الموظف والراتب فقط (وهي مربوطة فعليًا بالقيود المحاسبية بشكل صحيح)، لكن لا يوجد أي نموذج أو واجهة لطلبات الإجازة أو تسجيل الحضور.
التأثير	من أهم الوظائف اليومية لأي نظام موارد بشرية، غيابها الكامل يعني الاعتماد على أدوات خارجية موازية.
الحل المقترح	إضافة نموذج LeaveRequest بسير اعتماد بسيط (طلب → موافقة مدير → تحديث الرصيد)، وإن أمكن ربط لاحق بجهاز بصمة/حضور خارجي عبر API.

منخفض صفحة "الحالات" العامة تكرر ما هو ظاهر داخل كل قطاع بدون ربط

المشكلة	dashboard/cases/ تعرض نفس الحجوزات/الحالات الظاهرة مفلترة داخل كل صفحة قطاع (سفر، فنادق، إلخ) لكن بدون رابط مباشر بين العرضين.
التأثير	خطر عرض غير متسق لنفس البيانات في مكانين مختلفين بمرور الوقت لو تغيّر أحد المسارين ولم يتغيّر الآخر بنفس الطريقة.
الحل المقترح	توحيد مصدر البيانات (نفس ال query أو رابط مباشر "عرض في القطاع") بحيث تكون صفحة الحالات العامة بوابة موحدة بدل نسخة موازية.

8. الموافقات، الفوترة، السوق، الأخبار

حرج محرك الموافقات غير مربوط بأي وحدة أخرى في النظام

المشكلة	محرك الموافقات نفسه مبني بشكل جيد (فصل المهام، كشف التلاعب)، لكن لا يوجد أي مكان في النظام يستدعيه تلقائيًا — إنشاء أمر شراء كبير أو قيد محاسبي لا يُطلق أي طلب موافقة، والوصول الوحيد له هو استدعاء ال API مباشرة.
التأثير	أي قاعدة رقابة داخلية ("أي مصروف فوق كذا يحتاج موافقة مدير") غير قابلة للتطبيق فعليًا في الوقت الحالي رغم وجود كل البنية التحتية اللازمة لها.
الحل المقترح	ربط محرك الموافقات ك hook إلزامي عند نقاط حرجة محددة (اعتماد أمر شراء أعلى من حد معين، ترحيل قيد يدوي كبير، إلخ)، بحيث يوقف العملية فعليًا حتى تصدر الموافقة.

منخفض السوق (Marketplace): تفعيل حقيقي للقطاعات لكن بدون أي بيع/ترقية

المشكلة	require_plugin () يتحكم فعليًا في الوصول لل API (وليس مجرد شكل)، لكن لا يوجد أي منطق شراء أو ترقية مدفوعة من داخل السوق نفسه.
التأثير	التفعيل للقطاعات يبدو أنه يتم يدويًا/إداريًا فقط، وليس عبر تدفق تجاري ذاتي الخدمة للعميل.
الحل المقترح	إن كان الهدف نموذج SaaS ذاتي الخدمة، يحتاج ربط زر "تفعيل القطاع" في السوق بتدفق دفع/اشتراك فعلي (مرتبط بنفس نظام Paymob المستخدم في الفوترة).

9. القطاعات الأربعة (السياحة، الضيافة، تأجير السيارات، التوظيف)

السياحة: خطأ حقيقي في حساب هامش الربح — يظهر "غير محدد" دائمًا حرج

المشكلة دالة `calculate_booking_financials()` في الباكت `TravelFinancials` تُرجع حقولاً باسم `buy_price/sell_price/margin/commission` بينما الواجهة الأمامية (`TravelFinancials`) تتوقع أسماء مختلفة تمامًا: `/total_buy_price/total_sell_price` `.total_margin/margin_pct/commission_amount`.

التأثير مؤشر الهامش/الربح لكل حجز سياحي يظهر فارغاً (`undefined`) دائمًا في الواجهة رغم أن الحساب الفعلي صحيح في الباكت. إند — بيانات مالية مهمة مخفية بالكامل بسبب خطأ تسمية بسيط.

الحل المقترح توحيد أسماء الحقول بين الباكت إند والواجهة (إما تعديل استجابة الـ API أو تعديل الواجهة لتقرأ الأسماء الصحيحة) — إصلاح سريع نسبيًا وعالي الأثر.

السياحة: VendorRateCard موجود لكن غير مستخدم فعليًا متوسط

المشكلة نموذج أسعار الموردين المتعاقد معهم موجود في قاعدة البيانات، لكن كود إنشاء الحجز لا يقرأه أبدًا — التكلفة تُدخل يدويًا في كل مرة.

التأثير قدرة حقيقية مبنية لكنها معطلة عمليًا؛ فرصة ضائعة لتسريع إدخال الحجزات وتقليل أخطاء التسعير اليدوي.

الحل المقترح ربط شاشة إنشاء الحجز بقراءة `VendorRateCard` تلقائيًا لاقتراح التكلفة حسب المورد والخدمة المختارة.

السياحة: لا يوجد رفع مستندات (جواز/فيزا) ولا قائمة ركاب منظمة متوسط

المشكلة لا مكان لرفع صور جواز السفر أو الفيزا، وبيانات الركاب مخزنة كمصفوفة JSON غير منظمة بدلاً من نموذج بيانات فعلي.

التأثير صعوبة تتبع مستندات كل راكب، وصعوبة استخراج تقارير أو بحث دقيق عن راكب معين عبر الحجزات.

الحل المقترح إضافة مرفقات لكل حجز، وتحويل بيانات الركاب لنموذج (جدول) منظم بدل JSON حر.

الضيافة: لا يوجد تتبع حالة التنظيف (نظيف/متسخ/تم الفحص) للغرف متوسط

المشكلة لا يوجد حقل أو شاشة لحالة عمليات التدبير المنزلي (`housekeeping`) لكل غرفة.

التأثير لا يمكن معرفة جاهزية الغرفة فعليًا للحجز التالي بدون تواصل يدوي خارج النظام.

الحل المقترح إضافة حالة تنظيف بسيطة (نظيف/قيد التنظيف/تم الفحص) لكل غرفة، تتغير تلقائيًا عند تسجيل المغادرة وتُحدَّث يدويًا من فريق التدبير المنزلي.

الضيافة: لا يوجد تقرير إشغال (Occupancy) أو RevPAR متوسط

المشكلة لا توجد أي شاشة لعرض نسبة الإشغال أو مؤشر الإيراد لكل غرفة متاحة (`RevPAR`)، رغم توفر بيانات الحجزات الكافية لحسابها.

التأثير إدارة الفندق لا تملك رؤية أداء أساسية (الإشغال والإيراد) بدون حساب يدوي خارج النظام.

الحل المقترح بناء تقرير إشغال يومي/شهري ومؤشر `RevPAR` من بيانات الحجزات الموجودة فعليًا.

الضيافة: لا يوجد تسعير موسمي — سعر ثابت فقط منخفض

المشكلة لا خطط تسعير حسب الموسم أو اليوم من الأسبوع؛ السعر ثابت لكل غرفة.

التأثير فقدان مرونة تسعير أساسية في صناعة الفنادق (التسعير الديناميكي حسب الموسم/الطلب).

الحل المقترح إضافة نموذج "خطة تسعير" بحد أدنى (سعر أساسي + نسب تعديل حسب فترات محددة).

متوسط تأجير السيارات: لا توجد جدولة صيانة

المشكلة حقل حالة الصيانة موجود في النموذج لكنه لا يُضبط أو يُقرأ أبدًا من أي مكان في الكود — لا تتبع لمواعيد الصيانة الدورية. خطر تشغيل سيارات متأخرة عن الصيانة الدورية بدون أي تنبيه من النظام.

التأثير

الحل المقترح إضافة جدول صيانة بمواعيد مبنية على الكيلومترات/التاريخ، مع منع حجز أي سيارة تجاوزت موعد صيانتها المستحقة.

متوسط تأجير السيارات: أضرار الفحص عند التسليم لا تُحوّل تلقائيًا لفاتورة

المشكلة الفحص المزدوج (عند التسليم والاستلام) يكشف الفروقات (أضرار) فعليًا، لكن تحويلها لبند فاتورة يتطلب إدخالًا يدويًا منفصلًا.

التأثير

خطوة يدوية إضافية عرضة للنسيان تفوّت على الشركة تحصيل تكلفة الأضرار من العميل.

الحل المقترح ربط نتيجة الفحص المكتشفة تلقائيًا (بفروقاتها) بإنشاء بند فاتورة إضافي مقترح يعتمد على الموظف بضغطة واحدة.

حرج التوظيف والاستقدام: لا يوجد تتبع للتأشيرة/تصريح العمل — أهم فجوة في هذا القطاع

المشكلة لا توجد أي مرحلة أو حقل مخصص لتتبع حالة التأشيرة أو تصريح العمل ضمن سير عمل حالة التوظيف، رغم أن هذا هو جوهر عمل "الاستقدام" (تصدير عمالة دولية) تحديدًا.

التأثير

أهم عنصر تشغيلي في هذا القطاع (متابعة حالة التأشيرة من التقديم للإصدار) غير موجود في النظام إطلاقًا — يُدار بالكامل خارجه رغم أنه سبب وجود القطاع أصلًا.

الحل المقترح إضافة مرحلة/مراحل مخصصة لحالة التأشيرة ضمن محرك الحالات (تقديم → مراجعة طبية → موافقة → إصدار)، مع حقول تاريخ الصلاحية والتنبيه قبل الانتهاء.

عالي التوظيف: لا توجد مرحلة مقابلة رسمية، ولا إدارة عرض عمل منفصلة، ولا رفع مستندات

المشكلة لا يوجد جدولة مقابلات كمرحلة منفصلة في سير العمل، ولا مرحلة مخصصة لإدارة عرض العمل (Offer)، ولا مكان لرفع جواز السفر/الشهادة الطبية/العقد.

التأثير

دورة التوظيف الكاملة (من التقديم للتعيين الفعلي) غير مكتملة التمثيل في النظام — أجزاء أساسية منها تُدار خارجه.

الحل المقترح إضافة مراحل "مقابلة مجدولة" و"عرض عمل" كخطوات صريحة في محرك الحالات، وإضافة مرفقات لكل حالة توظيف.

متوسط التوظيف: لا يوجد نموذج بيانات مخصص للمرشح (Candidate)

المشكلة المرشح ممثّل حاليًا كـ Case عام ببيانات JSON غير منظمة بدلاً من نموذج بيانات مخصص له. صعوبة البحث والتقارير عن المرشحين (مثلاً: كل المرشحين لجنسية معينة أو مهنة معينة) لأن البيانات غير منظمة في أعمدة قابلة للاستعلام.

التأثير

الحل المقترح إنشاء نموذج Candidate مخصص بحقول منظمة (الجنسية، المهنة، المؤهل، إلخ) مرتبط بالـ Case بدل الاعتماد الكامل على JSON حر.

10. الإعدادات وال Onboarding

متوسط لا يوجد محرر لبيانات الشركة أو الإعدادات الضريبية

المشكلة صفحة الإعدادات تحتوي فقط على تاب عرض بروفايل للقراءة فقط، وتاب واتساب. لا يوجد مكان لتعديل اسم الشركة، الرقم الضريبي، العنوان، أو إعدادات الضرائب.

التأثير أي تعديل لهذه البيانات الأساسية (وهي تتغير أحيانًا) يتطلب تدخلًا مباشرًا في قاعدة البيانات.

الحل المقترح إضافة تاب "بيانات الشركة" قابل للتعديل، وتاب "إعدادات الضرائب" (نسب الضريبة الافتراضية، الرقم الضريبي المستخدم في فواتير ETA).

منخفض Onboarding خطوة واحدة فقط (اختيار القطاعات)

المشكلة لا يوجد wizard كامل لإعداد الشركة بعد التسجيل (دليل حسابات، إعدادات ضريبية، بيانات أساسية) — فقط اختيار القطاعات المفعل.

التأثير المستأجر الجديد يبدأ بحد أدنى من الإعداد، وقد يفوت خطوات مهمة (كإعداد دليل الحسابات الصحيح لنشاطه) دون توجيه واضح.

الحل المقترح توسيع ال onboarding لخطوات إضافية اختيارية (مراجعة/تخصيص دليل الحسابات المبدئي، بيانات الشركة الأساسية، إعداد أول مستخدم إضافي) — مع الإبقاء على OnboardingChecklist الحالي الذي يعمل بشكل جيد.

11. بوابة العملاء، الإشعارات، عناصر الهيدر

عالي بوابة العملاء (Portal): باك إند كامل بدون أي واجهة تستخدمه

المشكلة الباك إند يدعم فعليًا: دخول العميل بـ OTP، عرض فواتيره الخاصة، ودفع (وهي حاليًا عبر رابط Paymob placeholder). لكن لا توجد أي صفحة في الواجهة الأمامية تستهلك هذا الـ API إطلاقًا.

التأثير قدرة كاملة ومبنية (تجربة عميل ذاتية الخدمة لعرض ودفع فواتيره) معطلة بالكامل وغير مستغلة رغم جاهزية الباك إند.

الحل المقترح بناء تطبيق/صفحات بوابة عملاء بسيطة (دخول OTP، قائمة الفواتير، تفاصيل الفاتورة، رابط الدفع)، وربط الدفع الوهمي بحساب Paymob حقيقي عند الجاهزية.

متوسط لا يوجد نظام إشعارات داخل التطبيق — الجرس شكلي بالكامل

المشكلة أيقونة الجرس في الهيدر لا تملك أي onClick أو قائمة منسدلة أو استدعاء API — مجرد شكل بصري. كذلك عدد محفزات الإشعارات الفعلية الآن اثنان فقط (فاتورة مُرّحلة، فاتورة متأخرة)؛ لا تنبيه لمواعيد استحقاق الشيكات ولا لرفض فواتير الـ ETA.

التأثير أحداث مهمة (فاتورة إلكترونية مرفوضة، شيك مستحق) تمر دون أن يلاحظها أحد إلا بالمراجعة اليدوية.

الحل المقترح بناء نظام إشعارات داخل التطبيق (جدول إشعارات + قائمة منسدلة فعلية عند الجرس)، وتوسيع المحفزات لتغطي رفض ETA واستحقاق الشيكات على الأقل.

منخفض شريط البحث في الهيدر شكلي بالكامل

المشكلة لا يوجد submit handler ولا أي endpoint بحث مرتبط بشريط البحث الظاهر في أعلى كل صفحة. توقع مستخدم طبيعي ("يمكنني البحث من هنا عن أي شيء") لا يتحقق فعليًا، مما قد يسبب حيرة.

الحل المقترح إما ربطه ببحث عام حقيقي (عبر endpoint بحث موحد عبر الفواتير/العملاء/الحالات)، أو إخفاؤه مؤقتًا حتى يُبنى فعليًا.

لا توجد قائمة منسدلة فعلية للصورة الشخصية

منخفض

المشكلة الضغط على الصورة الشخصية في الهيدر لا يفعل شيئاً؛ زر تسجيل الخروج زر ثابت في الشريط الجانبي وليس داخل أي قائمة. **التأثير** لا مكان مركزي متوقع لإدارة الحساب (تعديل بروفایل، تغيير كلمة مرور، الخروج) رغم أنه نمط شائع ومتوقع في كل الأنظمة المشابهة.

الحل المقترح إضافة قائمة منسدلة حقيقية عند الضغط على الصورة الشخصية تجمع: تعديل البروفايل، تغيير كلمة المرور (بعد بنائها)، وتسجيل الخروج.

12. البنية التحتية: الاختبارات، CI/CD، الأنظمة الأساسية، الجاهزية للإنتاج

اختبار التزامن الكبير (50 ألف عملية) لا يعمل في أي مكان

متوسط

المشكلة `tests/load/test_50k_costing_concurrency.py` مستبعد من كل من إعدادات `pytest` الافتراضية وخطوات CI — لا يُشغّل تلقائياً إطلاقاً.

التأثير أخطر اختبار في المشروع (تزامن حساب التكلفة تحت ضغط عالي) غير مُتحقق منه أبداً، وأي كسر فيه لن يُكتشف تلقائياً.

الحل المقترح إضافة خطوة CI منفصلة (يمكن أن تكون مجدولة أسبوعياً بدل كل `push` لتقليل زمن التشغيل) تُشغّل هذا الاختبار تحديداً على بيئة قواعد بيانات حقيقية.

لا توجد خطوة CI للفرونت إند (Next.js) إطلاقاً

متوسط

المشكلة الـ `pipeline` الحالي يغطي `pytest` + `mypy` + `lint` للباك إند فقط، بدون أي خطوة `lint` أو `build` أو اختبار للواجهة الأمامية. **التأثير** أخطاء بناء الواجهة (بالضبط مثل أخطاء الاستيراد بحالة الأحرف اللبني أصلحناها فعلاً في هذا المشروع) لا تُكتشف تلقائياً قبل الدمج — تُكتشف فقط عند التشغيل الفعلي.

الحل المقترح إضافة خطوة CI مستقلة تُشغّل `npm run lint` و `npm run build` على الأقل عند كل `Pull Request`.

أخطاء `mypy` لا تمنع نجاح الـ CI (continue-on-error: true)

منخفض

المشكلة فحص الأنواع الثابتة (`mypy`) يعمل لكن فشله لا يوقف الـ `pipeline`.

التأثير أخطاء الأنواع تتراكم بصمت بمرور الوقت دون أن تُجبر الفريق على معالجتها.

الحل المقترح بعد معالجة الأخطاء الحالية الموجودة، تفعيل `continue-on-error: false` لجعل `mypy` فحصاً حقيقياً يمنع الدمج عند الفشل.

/app/core/audit كود مهجور — لا يوجد سجل تدقيق عام حقيقي

متوسط

المشكلة يوجد نموذج بيانات لسجل تدقيق عام، لكن بدون أي طبقة خدمة (`service`) وبدون أي مكان في الكود يستدعيه — موثّق صراحة في الكود نفسه بأنه "محجوز للاستخدام المستقبلي".

التأثير لا يوجد سجل تدقيق شامل يغطي كل النظام (فقط سلسلة الهاش الخاصة بالمحاسبة وحدها هي آلية التدقيق الحقيقية الوحيدة الموجودة فعلياً) — أي تعديل حساس خارج المحاسبة (مثل تغيير بيانات مستخدم أو حذف سجل) غير مسجّل في أي مكان.

الحل المقترح إما تفعيل هذا النظام فعلياً (بناء طبقة الخدمة وربطها كـ `middleware/hook` على العمليات الحساسة)، أو إزالته إن لم يكن هناك خطة قريبة لاستخدامه حتى لا يعطي انطباعاً خاطئاً بوجود تدقيق شامل.

متوسط

DEBUG=true بدون حماية من التشغيل بهذا الوضع في الإنتاج

المشكلة على عكس SECRET_KEY و CORS_ORIGINS اللذين لديهما فحص صريح يمنع القيم غير الآمنة في بيئة الإنتاج، متغير DEBUG لا يملك أي فحص مماثل، وقيمته الافتراضية في env.example هي true.

التأثير خطر تشغيل النظام في الإنتاج بوضع تصحيح مفضل (قد يُظهر تفاصيل حساسة في رسائل الخطأ) دون قصد.

الحل المقترح إضافة فحص عند بدء التشغيل يمنع DEBUG=true عندما ENVIRONMENT=production، بنفس منطق الفحص المطبق بالفعل على SECRET_KEY.

منخفض

خطر توافق في إصدارات الاعتماديات بالفرونت إند

المشكلة next مثبتت على "16.2.10" (إصدار حديث جدًا/تجريبي)، بينما eslint-config-next لا يزال على "14.2.35"، و react-dom مثبتتان على "18" رغم أن Next 16 عادة ما يتوقع React 19.

التأثير خطر كامن لمشاكل توافق أو تحذيرات/أخطاء غير متوقعة قد تظهر فجأة عند تحديث أي حزمة من هذه الثلاث بشكل منفصل.

الحل المقترح تحديث eslint-config-next و react/react-dom لتتوافق فعليًا مع إصدار Next.js المستخدم، أو تثبيت Next.js على إصدار مستقر (LTS) بدل الإصدار التجريبي الأحدث.

13. أمان تقني إضافي (فحص عميق)

SQL injection، رفع الملفات، توقيع الـ CORS، XSS، webhooks، تسريب بيانات بين المستأجرين

حرج

تسجيل دخول بوابة العملاء بدون أي تحقق حقيقي من الهوية

المشكلة شاشة دخول العميل في بوابة العملاء (Portal) مكتوب في الكود نفسه بالحرف الواحد إنها "Mock OTP" مع تعليق صريح: "في الإنتاج ده المفروض بيعت ويتحقق من OTP حقيقي". فعليًا، النظام يصدر توكن دخول صالح لمجرد إدخال رقم تليفون العميل ومعرّف المستأجر — من غير أي كلمة سر أو رمز تحقق.

التأثير أي حد يعرف رقم تليفون عميل (مش سر) ومعرّف المستأجر (يظهر في الروابط/الإيميلات) يقدر ياخذ توكن دخول صالح 24 ساعة لحساب هذا العميل، ويشوف فواتيره وكشف حسابه المالي. الخطر حاليًا محدود لأن مفيش واجهة أمامية فعلية تستخدم البورتال بعد (زي ما اتقال في التقرير الأساسي)، لكن يجب إصلاحه قبل بناء أي واجهة عليه.

الحل المقترح ربط الشاشة بإرسال والتحقق من رمز OTP حقيقي عبر SMS أو واتساب (البنية التحتية لواتساب موجودة بالفعل في النظام) قبل إصدار أي توكن دخول، أو تعطيل مسار الدخول هذا مؤقتًا حتى يتم بناء التحقق الحقيقي.

حرج

الـ webhooks (الشحن وواتساب) تقبل بيانات مزورة لو غاب التوقيع الأمني تمامًا

المشكلة التحقق من توقيع الأمان (signature) في webhooks بوستا، ماليرز، وواتساب مكتوب بحيث لو الطلب الوارد ما في هوش هيدر التوقيع خالص، الكود بيعتبره "موثوقًا" تلقائيًا ويكمل التنفيذ، بدل ما يرفضه. التحقق شغال فقط لو المُرسل "اتفضل" وبعث توقيع أصلاً.

التأثير أي طرف خارجي يقدر بيعت طلبًا مباشرًا لأي endpoint من دول (مثلاً "الشحنة اترجعت" أو رسالة واتساب واردة) لأي مستأجر معروف معرفه، من غير أي إثبات هوية، والنظام هياخد البيانات المزورة دي على إنها حقيقية ويحدث بيها حالة الشحنة أو بيانات العميل. هذا نمط ثغرة أمنية كلاسيكي (fail-open بدل fail-closed). ملحوظة: نفس هذه النقطة مُطبقة صَح في webhook الدفع الخاص بـ Paymob، وهو النموذج الصحيح الذي يجب تعميمه.

الحل المقترح تغيير المنطق في الثلاث دوال ليرفض الطلب فورًا لو التوقيع غائب تمامًا (وليس فقط لو كان موجودًا وغير صحيح) — تمامًا بنفس الطريقة المطبقة بالفعل في تحقق Paymob.

بناء استعلام SQL يدوي في مسار واحد فقط بدل الآلية الموحدة الآمنة

منخفض

المشكلة كل النظام تقريبًا يستخدم آلية موحدة وآمنة لعزل بيانات كل مستأجر (schema_translate_map عبر SQLAlchemy)، ما عدا شاشة تسجيل دخول بوابة العملاء التي تبني جملة SQL يدويًا (SET search_path) بدل استخدام هذه الآلية. حاليًا غير قابل للاستغلال لأن القيمة المُستخدمة مضبوطة كمعروف UUID، لكنه نمط مختلف وغير موحد عن باقي الكود.

التأثير لا يوجد خطر فعلي حاليًا، لكن أي نسخ لهذا الكود مستقبلاً لمكان يقبل نصًا حرًا بدل UUID قد يفتح ثغرة حقن SQL حقيقية على مستوى schema كامل.

الحل المقترح توحيد هذا الجزء ليستخدم نفس الآلية الآمنة المستخدمة في باقي النظام بدل إعادة بناء نفس المنطق يدويًا.

14. قاعدة البيانات والجاهزية للإنتاج

الحذف الناعم، النسخ الاحتياطي، حماية التكرار، إعدادات Docker

سكربت استعادة النسخة الاحتياطية (Restore) عبارة عن محاكاة فقط ولا يستعيد أي شيء فعليًا

حرج

المشكلة السكربت المخصص لاختبار استعادة نسخة احتياطية لمستأجر (tenant_restore.py) طلع إن أمر الاستعادة الفعلي (pg_restore) معلق كتعليق كود (commented out) واستُبدل بسطر واحد بيطبع "تمت المحاكاة بنجاح". السكربت بعدها يشغّل فحص سلامة بيانات على القاعدة الموجودة أصلاً، فييدي انطباع كاذب بنجاح عملية الاستعادة.

التأثير لو الشركة اعتمدت على تشغيل هذا السكربت للتأكد إن النسخ الاحتياطي شغال (drill)، هيظهر النتيجة "ناجحة" دائماً حتى لو مفيش نسخة احتياطية حقيقية تُستعاد أصلاً — وهذه أخطر من عدم وجود نسخ احتياطي خالص، لأنه بيدي ثقة زائفة.

الحل المقترح تفعيل أمر pg_restore الحقيقي (أو الربط بآلية استعادة قاعدة البيانات السحابية المُستخدمة فعليًا — Neon)، والتأكد إن فحص السلامة بعد الاستعادة بيتحقق من نسخة مُستعادة فعليًا وليس القاعدة الأصلية.

لا يوجد أي استراتيجية نسخ احتياطي موثقة أو مجدولة

عالي

المشكلة لا توجد أي إشارة في المشروع (سكربتات، توثيق، إعدادات) لجدولة نسخ احتياطي دوري لقاعدة البيانات السحابية.

التأثير في حال حدوث خطأ بشري (حذف بيانات بالغلط) أو مشكلة تقنية كبيرة، لا توجد خطة تعافي واضحة أو مجدولة.

الحل المقترح توثيق والاعتماد على آلية النسخ الاحتياطي التلقائي لموفر قاعدة البيانات السحابية المستخدم (Neon يوفر point-in-time recovery)، والتأكد من ضبطها فعليًا وتجربتها دوريًا.

لا يوجد حماية من التكرار عند الدفع/الفوترة (Idempotency) — خطر دبل-كليك

عالي

المشكلة شاشات إنشاء فاتورة من نقطة البيع، وتسجيل دفعة لمورد، ليس لديها أي آلية تمنع تنفيذ نفس الطلب مرتين لو المستخدم ضغط الزرار مرتين أو حصل تأخير في الشبكة وأعاد المتصفح الإرسال تلقائيًا.

التأثير دبل-كليك واحد بالغلط من كاشير مشغول ممكن ينشئ فاتورتين وقيدتين محاسبين مكررين بدل واحد، وهو خطأ شائع الحدوث عمليًا وصعب اكتشافه لاحقًا بدون تدقيق يدوي.

الحل المقترح إضافة مفتاح تكرار (Idempotency Key) يُرسل مع كل طلب إنشاء فاتورة/دفعة حساس، بحيث لو نفس المفتاح اتكرر يرجع النظام نفس النتيجة الأصلية بدل تنفيذ العملية مرة ثانية.

حذف نهائي بدون أثر لبيانات مالية في قطاع السياحة (باقات وتأشيرات)

متوسط

المشكلة عمود "حذف ناعم" (deleted_at) موجود فعليًا في كل جداول النظام لكنه غير مُستخدَم في أي مكان خالص — بينما حذف باقة سياحية أو طلب تأشيرة يتم كحذف نهائي فعلي (hard delete) من قاعدة البيانات بلا أي أثر، رغم أن هذه السجلات تحمل بيانات تكلفة/رسوم مالية.

التأثير لا يوجد أي سجل أو أثر لو تم حذف باقة أو تأشيرة بالغلط أو بقصد سيء — البيانات تختفي نهائيًا بدون إمكانية استرجاع أو تدقيق.

الحل المقترح تحويل حذف الباقات والتأشيرات لنفس نمط "إلغاء تنشيط" (is_active) المُستخدَم بالفعل في وحدتي الفنادق وتأجير السيارات، بدل الحذف الفعلي من القاعدة.

لا يوجد إعداد Docker جاهز للإنتاج — حتى تشغيل الباك إند داخل حاوية غير متاح حاليًا

متوسط

المشكلة ملف docker-compose.yml الحالي مخصص للتطوير المحلي فقط، وخدمة الباك إند نفسها أُزيلت منه بالكامل (تعليق في الملف نفسه يشرح السبب: تعارضها مع إعدادات التطوير الفعلية). الـ Dockerfile الموجود (Dockerfile.dev) مخصص للتطوير فقط (تثبيت قابل للتعديل، بدون مستخدم غير جذر، بدون أمر تشغيل نهائي).

التأثير لا يوجد حاليًا أي طريقة لتشغيل الباك إند داخل حاوية Docker جاهزة للإنتاج — نشر النظام فعليًا سيحتاج بناء هذا الإعداد من الصفر وقت الحاجة له.

الحل المقترح بناء Dockerfile إنتاجي منفصل (multi-stage build، بدون صلاحيات جذر، بدون أدوات تطوير) وملف docker-compose أو إعداد نشر مخصص للإنتاج مع حدود موارد وسياسة إعادة تشغيل مناسبة.

15. تجربة المستخدم، الأداء، وإتاحة الاستخدام

بيانات وهمية في الداشبورد، الطباعة والتصدير، الترميم، الموبايل، الإتاحة

الداشبورد الرئيسي يعرض أرقامًا ورسومًا بيانية وهمية بجانب البيانات الحقيقية

عالي

المشكلة في الصفحة الرئيسية للوحة التحكم، كارت "عدد الطلبات" مثبت دائمًا على الرقم 1,842 وكارت "العملاء الجدد" مثبت على 312 — أرقام ثابتة في الكود مش قادمة من قاعدة البيانات خالص. كذلك الرسوم البيانية الثلاثة (الإيرادات، توزيع المصروفات، أداء الفروع) بيانات وهمية بالكامل، بما فيها أسماء فروع غير موجودة أصلاً ("فرع القاهرة"، "فرع الإسكندرية"، "فرع الجيزة").

التأثير صاحب الشركة أو المدير يشوف كروت ورسوم بيانية شكلها احترافي ومقنع تمامًا، لكنها مش بتعكس أي بيانات حقيقية عن نشاط الشركة الفعلي — خطر فهم غلط لأداء الشركة الحقيقي بالاعتماد على أرقام وهمية.

الحل المقترح إما ربط هذه الكروت والرسوم البيانية ببيانات حقيقية من قاعدة البيانات، أو وضع علامة واضحة "بيانات تجريبية" عليها مؤقتًا لحد ما تتفعل فعليًا، لمنع أي التباس.

لا يوجد طباعة أو تصدير PDF/Excel لأي فاتورة أو تقرير في كل النظام

عالي

المشكلة الاستثناء الوحيد الموجود هو تصدير CSV بسيط جدًا لأربع أرقام فقط من كروت الداشبورد. لا توجد أي إمكانية طباعة أو تصدير PDF لفاتورة مبيعات، فاتورة مشتريات، إيصال نقطة بيع، أو أي تقرير في النظام كله.

التأثير من أبسط احتياجات أي نظام ERP يومية — العميل يطلب فاتورة PDF يبعثها له، أو المحاسب يحتاج يطبع تقرير. غيابها التام يعني الاعتماد على أدوات خارجية أو لقطات شاشة.

الحل المقترح إضافة توليد PDF لفواتير المبيعات/المشتريات وإيصالات نقطة البيع كأولوية أولى (يمكن استخدام مكتبة توليد PDF من القالب مباشرة في الباك إند)، ثم توسيعها لباقي التقارير الرئيسية.

قوائم كثير في الواجهة بتحمّل كل البيانات دفعة واحدة بدون ترقيم صفحات

متوسط

المشكلة صفحات الحالات (Cases)، أصناف المخزون، الموردین، وفواتير المبيعات كلها تتطلب كل السجلات من الـ API دفعة واحدة بدون أي حد أقصى (limit) أو ترقيم صفحات (pagination) أو بحث/فلتر من جانب السيرفر. صفحة جهات الاتصال فيها ترقيم من ناحية الباك إند لكن بدون أي زرار "صفحة تالية" فعلي في الواجهة يستخدمه.

التأثير هذه الصفحات هتبطأ بشكل ملحوظ كل ما البيانات تكبر مع الوقت (آلاف الفواتير أو الأصناف)، وقد تصل لحد لا يمكن تحميله في المتصفح أصلاً — من أوضح مخاطر الأداء المكتشفة في الفحص كله.

الحل المقترح توحيد كل قوائم النظام على نفس نمط الترقيم من جانب السيرفر (server-side pagination) مع بحث وفلتر حقيقيين، وإكمال ربط الواجهة بهذا الترقيم في صفحة جهات الاتصال تحديداً.

شاشة نقطة البيع غير متجاوبة مع شاشات الموبايل/التابلت الصغيرة

متوسط

المشكلة تخطيط شاشة نقطة البيع مبني بعرض ثابت (380px لسلة المشتريات) بدون أي تكيف مع الشاشات الصغيرة، بينما باقي صفحات النظام (الداشبورد، المحاسبة، السياحة) متجاوبة بشكل جيد.

التأثير الكاشير غالباً يستخدم تابلت أو شاشة صغيرة في نقطة البيع الفعلية — تخطيط ثابت زي ده هيتكسر أو يبقى صعب الاستخدام على هذا النوع من الأجهزة تحديداً.

الحل المقترح تعديل تخطيط الشاشة ليتحول لعمود واحد (سلة المشتريات تحت الكتالوج) على الشاشات الصغيرة، ويرجع لعمودين على الشاشات الكبيرة فقط.

لا توجد أي عمليات جماعية (Bulk Actions) في كل النظام تقريباً

منخفض

المشكلة الاستثناء الوحيد هو رفع ملف إكسل لاستيراد مرشحين بالجملة في وحدة التوظيف. لا يوجد اختيار متعدد لصفوف في أي جدول، ولا إرسال جماعي لفواتير، ولا تحديث حالة جماعي، ولا حذف/تصدير جماعي في أي مكان.

التأثير أي عملية تخص عدد كبير من السجلات (مثلاً إرسال تذكير لكل الفواتير المتأخرة) لازم تُعمل سجل سجل يدوياً.

الحل المقترح إضافة اختيار متعدد (checkboxes) وعمليات جماعية أساسية (إرسال، تحديث حالة) في أكثر الصفحات استخداماً يومياً أولاً (الفواتير، الحالات).

أزرار كثير بأيقونة فقط بدون أي وصف نصي لذوي الإعاقة البصرية

منخفض

المشكلة من إجمالي 190 زرار في الواجهة، 4 فقط عليهم وصف نصي (aria-label) موزعين على 3 ملفات بس — يعني الغالبية العظمى من الأزرار (زرار الحذف، زرار إغلاق أي نافذة منبثقة، أزرار +/- في نقطة البيع) بدون أي وصف لقارئ الشاشة.

التأثير أي موظف يعتمد على تقنية مساعدة (قارئ شاشة) مش هيقدر يفهم وظيفة هذه الأزرار خالص.

الحل المقترح إضافة aria-label لكل الأزرار التي تحتوي على أيقونة فقط بدون نص مرئي بجانبها — تعديل ميكانيكي بسيط لكنه منتشر في كل الشاشات.

تنسيق التواريخ غير موحد عبر شاشات النظام (عربي في نص، إنجليزي في نص ثاني)

منخفض

المشكلة حوالي نصف الصفحات (الداشبورد، الموارد البشرية، الشيكات، الأخبار، جهات الاتصال) تعرض التاريخ بالتنسيق العربي، والنصف الآخر (الموافقات، الفوترة، تفاصيل الحالات، الفاتورة الإلكترونية، المشتريات، المبيعات) تعرضه بالتنسيق الإنجليزي.

التأثير تناقض بصري ملحوظ لنفس النوع من البيانات في نفس النظام حسب الصفحة اللي بتفتحها.

الحل المقترح توحيد كل الصفحات على دالة تنسيق تاريخ مشتركة واحدة بدل كل صفحة تعمل تنسيقها الخاص بشكل منفصل.

استقبال رسائل واتساب من العملاء موجود لكن غير مستخدم — الرسائل تُستقبل وتُهمل

منخفض

النظام فعليًا قادر على استقبال رسائل واتساب واردة من العملاء (التحقق من التوقيع، وربط الرسالة بالمستأجر والعميل الصحيح) لكن لا يوجد أي جزء في النظام يستمع لهذه الرسائل أو يتصرف بناءً عليها — يتم استقبالها وتحليلها ثم تجاهلها فعليًا.

المشكلة

قدرة حقيقية ومبنية (محادثة ثنائية الاتجاه عبر واتساب مع العملاء) معطلة بالكامل رغم أن أغلب المطلوب لتفعيلها جاهز فعليًا.

التأثير

ربط الرسائل الواردة بمحرك الحالات (Cases) كما هو موضح فعليًا في تعليق الكود نفسه، بحيث تتحول رسالة العميل الواردة لحدث فعلي يمكن التعامل معه (مثلًا فتح حالة جديدة أو الرد التلقائي).

الحل المقترح

خطة الأولويات المقترحة

ترتيب مقترح للبدء بالإصلاحات — من الأكثر أولوية للأقل

عاجل جدًا

1. تفعيل الصلاحيات (RBAC) فعليًا على مستوى الـ API والواجهة — أكبر فجوة أمان تمنع توزيع النظام على موظفين بأمان.
2. إصلاح تسجيل دخول بوابة العملاء (Mock OTP حاليًا بدون أي تحقق حقيقي من الهوية).
3. رفض أي webhook (شحن/واتساب) بدون توقيع أمني بدل قبوله تلقائيًا.
4. تفعيل زر إرسال الفاتورة الإلكترونية (ETA) في الواجهة — الباك إند جاهز 100%.
5. إصلاح ربط مصاريف الاستيراد بتكلفة المخزون (DossierCostLayer لا يُكتب أبدًا).
6. إصلاح خطأ تسمية الحقول في هامش ربح حجوزات السياحة (undefined دائمًا).
7. حذف أو إصلاح الكود الميت الخطر في contacts/service.py.
8. إضافة استعادة كلمة المرور (مفقودة تمامًا رغم أن SMTP جاهز).

عالي

1. تفعيل أمر الاستعادة الحقيقي في سكريبت النسخ الاحتياطي (حاليًا محاكاة فقط ومضللة).
2. إضافة حماية من التكرار (Idempotency) على الدفع/الفوترة لمنع أخطاء الدبل-كليك.
3. ربط كروت ورسوم الداشبورد الرئيسي ببيانات حقيقية بدل الأرقام الوهمية الحالية.
4. إضافة تصدير/طباعة PDF لفواتير المبيعات والمشتريات وإيصالات نقطة البيع على الأقل.
5. فترة القائمة الجانبية حسب القطاعات المفعل فعليًا لكل مستأجر بدل عرضها كلها دائمًا.
6. كشف التقارير المالية (ميزان المراجعة على الأقل) في واجهة المحاسبة.
7. ربط محرك الموافقات فعليًا بعمليات الشراء/القيود الكبيرة.
8. إضافة تتبع التأشيرة/تصريح العمل في قطاع التوظيف والاستقدام.
9. تدوير رمز التحديث (refresh token rotation) لتقليل خطر التسريب.

متوسط

1. توحيد كل قوائم النظام (الحالات، المخزون، الموردين، الفواتير) على ترقيم صفحات حقيقي من السيرفر.
2. إضافة أوامر الشراء وإذن الاستلام لتفعيل المطابقة الثلاثية الحقيقية.
3. إضافة واجهة تسجيل دفعات ومرتجعات المبيعات.
4. بناء واجهة رفع ومراجعة كشوف الحسابات لوحدة التسويات.
5. بناء نظام إشعارات داخل التطبيق (الجرس حاليًا شكلي).
6. إضافة رفع مستندات (جواز/فيزا/عقد) للسياحة والتوظيف.

7. تحويل حذف باقات/تأشيرات السياحة من حذف نهائي إلى إلغاء تنشيط.
8. بناء إعداد Docker إنتاجي للباك إند (غير موجود حاليًا حتى للتطوير).

عند التفرغ

1. جعل شاشة نقطة البيع متجاوبة مع شاشات الموبايل/التابلت.
2. إضافة aria-label لكل الأزرار ذات الأيقونة فقط (إتاحة الاستخدام لضعاف البصر).
3. توحيد تنسيق التواريخ عبر كل الشاشات (عربي أو إنجليزي، وليس مزيجًا).
4. ربط رسائل واتساب الواردة من العملاء بمحرك الحالات بدل تجاهلها.
5. إضافة عمليات جماعية أساسية (bulk actions) على الأقل للفواتير والحالات.
6. بناء واجهة أمامية لبوابة العملاء (الباك إند جاهز، بعد إصلاح آلية الدخول).
7. إضافة تقارير الإشغال/RevPAR للفنادق، والتسعير الموسمي.
8. إضافة جدول الصيانة لتأجير السيارات.
9. تفعيل خانة "تذكرني" أو إزالتها.
10. تحديث اعتماديات الفرونت إند لتتوافق مع بعضها.

هذا التقرير يوثق نتائج الفحص فقط دون أي تعديل على الكود، بناءً على طلب صريح بعدم التعديل حتى صدور تعليمات لاحقة. جاهز للاستخدام كخطة عمل عند البدء في التنفيذ.